

Security, Backend & Infrastructure

The security architecture of AROH is built on the principle that trust must never originate from the client. Every browser, mobile application, desktop application, API consumer, or third-party integration should be considered an untrusted environment regardless of how polished or controlled the user interface appears. The responsibility of validating identity, enforcing permissions, protecting financial integrity, managing sensitive data, and executing privileged operations belongs exclusively to the server-side platform. Client applications exist to present information and collect user intent; they should never become authoritative over business-critical state. This philosophy becomes increasingly important as the ecosystem expands because a security weakness within the platform affects every product simultaneously.

The backend should be viewed as the operational core of the platform rather than merely a collection of API endpoints. It coordinates business workflows, validates requests, enforces permissions, executes transactions, publishes events, records audit logs, communicates with infrastructure providers, and guarantees consistency across the ecosystem. Every operation that modifies business state should pass through this application layer. Wallet updates, membership changes, administrative actions, notification publishing, profile modifications, product management, and future billing operations should never bypass server-side validation regardless of the client from which they originate. The backend represents the authoritative implementation of business rules, ensuring that products remain lightweight consumers of platform capabilities rather than independent systems with duplicated logic.

Authentication should establish identity through cryptographically verifiable credentials rather than client-generated claims. Session management must support secure token lifecycle management, expiration, revocation, refresh mechanisms, device awareness, and future multi-factor authentication. Authorization should remain independent from authentication, evaluating permissions according to centrally managed policies rather than trusting client-side role information. Every privileged request should verify both identity and authorization before executing business logic. Administrative capabilities should require additional verification where appropriate, and sensitive operations should produce immutable audit records for accountability and forensic analysis.

Role-based access control should provide the initial authorization model while remaining flexible enough to evolve toward attribute-based and policy-based authorization as the ecosystem matures. Permissions should originate from centralized policy evaluation rather than hardcoded conditionals scattered throughout products. Every protected resource should define explicit access requirements, allowing products to request authorization decisions from the platform instead of independently evaluating permissions. This centralized approach simplifies maintenance, improves consistency, and reduces the likelihood of privilege escalation vulnerabilities.

The Aros Wallet represents one of the highest-security domains within the ecosystem because it governs financial integrity. Wallet balances, transaction ledgers, membership purchases, rewards, refunds, promotions, and future marketplace transactions should always remain server-authoritative. No client application should possess the ability to modify balances directly. Every financial operation should execute as an atomic transaction with complete validation, authorization, auditing, and rollback protection. Immutable ledger records should preserve historical accuracy, ensuring that balances can always be reconstructed from transaction history rather than relying solely on mutable state. Fraud prevention, rate

limiting, anomaly detection, and future financial compliance mechanisms should evolve around this authoritative transaction model.

API architecture should provide stable, well-documented contracts between products and platform services. APIs should expose business capabilities rather than database structures or infrastructure details. Input validation should occur immediately upon request reception using shared schemas, followed by authentication, authorization, business validation, transaction execution, event publication, structured response generation, and comprehensive logging. Every API should produce predictable error structures, versioned contracts, and consistent response formats. Public APIs, internal APIs, administrative APIs, and future partner integrations should all follow common architectural principles while maintaining appropriate security boundaries.

Data persistence should remain abstracted behind repository or service layers rather than exposing infrastructure-specific implementations throughout the ecosystem. Although Firebase currently serves as the initial backend provider, architectural abstractions should allow future migration to alternative technologies without requiring product-level modifications. Infrastructure decisions should remain isolated within platform modules, ensuring that changes to storage providers, authentication systems, caching technologies, search engines, or analytics platforms affect only the corresponding infrastructure layer rather than every consuming application.

Database architecture should prioritize data integrity, ownership clarity, and scalability. Every entity should belong to exactly one bounded context, with relationships defined through stable identifiers rather than tightly coupled object graphs. Normalization should be balanced against performance requirements while preserving maintainability and avoiding duplication of authoritative business data. Sensitive information should remain encrypted where appropriate, indexing strategies should support anticipated query patterns, and migrations should evolve incrementally with complete backward compatibility whenever possible. Data models should reflect business concepts rather than implementation convenience, ensuring that domain evolution remains independent of infrastructure changes.

Infrastructure should support multiple operational environments while maintaining strict separation between development, testing, staging, preview, and production deployments. Configuration should originate from secure environment management systems rather than application code, allowing operational changes without requiring recompilation or redeployment. Secrets must never be committed to version control, exposed within client bundles, or embedded into reusable packages. Environment validation should occur during startup so configuration errors are detected immediately rather than surfacing unpredictably during runtime.

Deployment architecture should emphasize reliability, repeatability, and observability. Continuous integration pipelines should validate formatting, linting, type safety, dependency integrity, automated testing, documentation consistency, and build correctness before changes reach production. Continuous deployment should support preview environments, staged rollouts, rollback capability, and deployment verification through automated health checks. Infrastructure should evolve toward immutable deployments where releases are promoted through environments rather than modified in place, reducing operational variability and simplifying incident recovery.

Observability should be treated as a core architectural capability rather than an operational afterthought. Logging, metrics, distributed tracing, health checks, performance monitoring, audit records, security

events, and business analytics should provide comprehensive visibility into platform behavior. Engineers should be able to diagnose issues through collected telemetry instead of relying upon manual reproduction. Monitoring should extend beyond infrastructure health to include authentication success rates, wallet transaction integrity, notification delivery, AI service performance, API latency, error rates, membership conversions, and future cross-product workflows. Operational dashboards should provide actionable insights rather than isolated technical metrics.

Performance engineering should focus on eliminating systemic bottlenecks rather than isolated optimizations. Caching strategies, asynchronous processing, background jobs, queue management, efficient database access, optimized asset delivery, and intelligent request handling should all contribute to predictable platform responsiveness. Expensive operations such as analytics aggregation, notification delivery, report generation, AI processing, and future marketplace synchronization should execute asynchronously whenever possible, allowing user-facing interactions to remain responsive. Performance improvements should always preserve correctness, maintainability, and security rather than introducing unnecessary architectural complexity.

Infrastructure resilience should assume that failures are inevitable. External providers may become unavailable, networks may experience latency, deployments may fail, and individual services may require maintenance. The platform should therefore incorporate graceful degradation, retry strategies, timeout management, circuit breaking where appropriate, comprehensive backups, disaster recovery planning, and clear operational procedures. No single infrastructure dependency should compromise the long-term stability of the ecosystem. As AROH evolves into a multi-product platform serving diverse users and future enterprise customers, backend architecture and infrastructure governance will increasingly become strategic assets. A secure, observable, resilient, and well-architected backend ensures that every present and future product inherits operational excellence from the platform, allowing innovation to occur at the product layer while the underlying foundation remains stable, trustworthy, and continuously scalable.